ngspice Simulator Internals

A ground-up guide to how ngspice-46 works inside, and how OpenVAF .osdi models plug in

Ngspice + OpenVAF Enhancements project

July 2026

Contents

ngspice Simulator Internals 2
- How to read this document . . . 3
- Chapter 1 — What ngspice is, and the shape of the problem . . . 4
- Chapter 2 — The system at a glance . . . 5
- Chapter 3 — The numerical background you'll need . . . 6
- Chapter 4 — The frontend: the command shell and control language . . . 7
- Chapter 5 — From netlist text to a circuit . . . 8
- Chapter 6 — The seam: `IFsimulator` / `IFfrontEnd` . . . 9
- Chapter 7 — The circuit in memory: `CKTcircuit` . . . 10
- Chapter 8 — The device interface: `SPICEdev` and the registry . . . 11
- Chapter 9 — The matrix and the solver: SMP → Sparse 1.3 / KLU . . . 12
- Chapter 10 — The Newton core: `CKTload`, `NIiter`, convergence . . . 13
- Chapter 11 — The analyses . . . 14
- Chapter 12 — The OSDI bridge: how OpenVAF models become devices . . . 15
- Chapter 13 — XSPICE: the event-driven engine and code models . . . 16
- Chapter 14 — The output data model: dvecs, plots, the mini-MATLAB . . . 17
- Chapter 15 — A complete worked example: an RC circuit end to end . . . 18
- Chapter 16 — Where this project touched ngspice . . . 19
- Chapter 17 — Reference appendices . . . 20
- Closing note . . . 21

ngspice Simulator Internals

A ground-up guide to how ngspice-46 works inside: how a netlist becomes a running circuit, how the analog engine solves it, and how OpenVAF-compiled (`.osdi`) models plug in. It is the simulator-side companion to the OpenVAF-r Compiler Internals guide — that one ends where a `.osdi` file is produced; this one begins where a simulator loads it.

How to read this document

You do not need to know SPICE internals already. Each chapter builds on the last. Chapters 1-3 set up the shape of the problem and the numerical background; 4-6 follow a netlist from text into the engine; 7-11 are the analog core (the circuit in memory, devices, the matrix, Newton's method, the analyses); 12-14 cover the two big extension mechanisms (OSDI and XSPICE) and the output/vector world; 15 traces one RC circuit end to end; 16 maps this project's enhancements onto the code; 17 is reference material.

Everything is grounded in the actual source under `ngspice-46/src/`. File and function names are real; follow the links to read the code. Where a line is *load-bearing* — a struct field, a function-pointer table, a mode flag — it is named explicitly so you can grep for it.

Chapter 1 — What ngspice is, and the shape of the problem

ngspice is an analog/mixed-signal circuit simulator descended from Berkeley SPICE3. Its job: take a netlist (a text description of components and how they connect) and compute the voltages and currents over a DC operating point, a frequency sweep, or a time interval.

The mathematical heart is Modified Nodal Analysis (MNA). Every circuit node gets an unknown voltage; every component contributes current equations by Kirchhoff's Current Law (KCL: currents into a node sum to zero). Stacking those equations gives a system

```
F(x) = 0
```

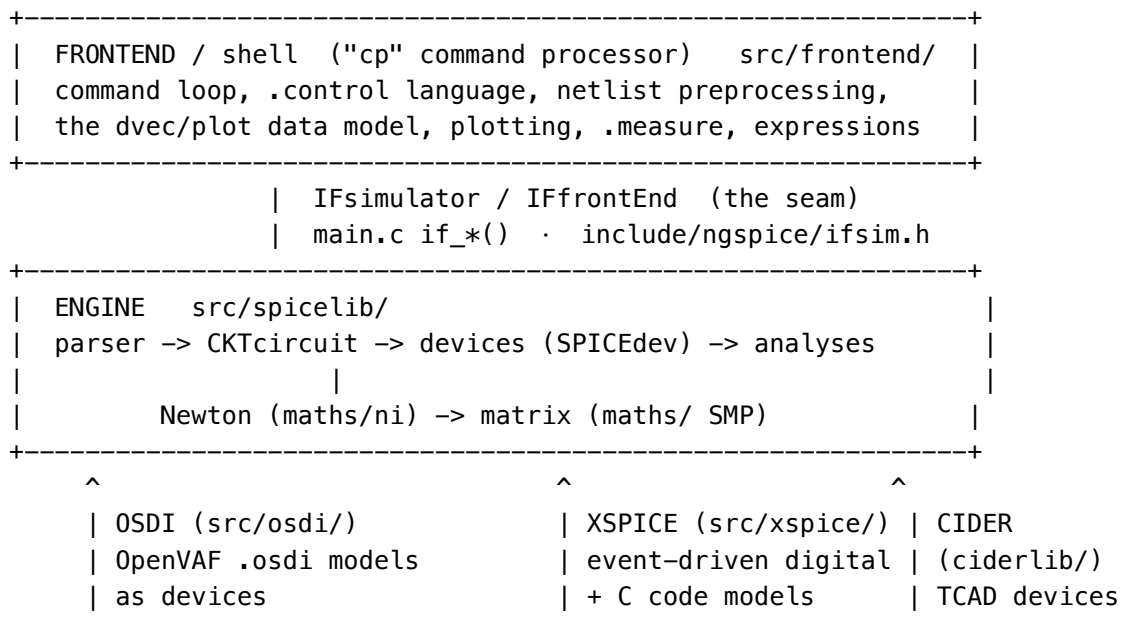
where `x` is the vector of node voltages (plus a few branch currents), and `F` is the net current mismatch at every node. For a *linear* circuit this is one matrix solve `G·x = b`. Real circuits are nonlinear (diodes, transistors), so ngspice solves `F(x) = 0` with Newton's method: repeatedly linearize (`J·Δx = −F`, where `J` is the Jacobian $\partial F/\partial x$), solve, update, until it converges. For transient analysis it does this at every timestep, with the capacitor/inductor charges discretized by a numerical integration rule.

So at bottom ngspice is a loop of: *build a sparse matrix and right-hand side from the components at the current guess → solve it → check convergence → repeat*. Almost everything else — the netlist language, the command shell, the device model library, the plotting — is scaffolding around that loop.

A key architectural fact: ngspice is two programs in one address space. A command-line shell (the interactive `ngspice>` prompt, the `.control` language, plotting, `.measure`) sits on top of a simulation engine (the parser, the circuit, the devices, the solver), and the two talk only through a narrow interface. That split is the first thing to understand.

Chapter 2 — The system at a glance

The three layers and the seam



The seam is `IFsimulator`/`IFfrontEnd` (defined in `ifsim.h`, implemented by the `if_*` functions in `main.c`). The shell never touches circuit internals directly; it calls `if_inpdeck` to parse a deck, `if_run` to run an analysis, `if_setparam` to alter a value. This is exactly what makes **`libngspice`** possible: `sharedspice.c` is a second front end that drives the same engine through the same seam.

The source tree by size

Subsystem	Lines	What it is
`spicelib/devices/`	~468K	58 device families — the compact-model library (BSIM, HICUM, PSP, VBIC, …). The bulk of ngspice.
`frontend/`	~85K	the command shell, netlist preprocessing, plotting, the dvec/plot data model
`maths/`	~43K	the matrix (Sparse 1.3 + KLU) and Newton-iteration core
`xspice/`	~38K	the event-driven mixed-signal engine + C code models
`ciderlib/`	~28K	numerical (TCAD) device simulation on a mesh
`spicelib/analysis/`	~18K	the analysis drivers (op, tran, ac, noise, pz, sens, …)
`include/ngspice/`	~17K	the shared headers and data structures
`spicelib/parser/`	~13K	the 3-pass netlist parser + behavioral expression trees
`osdi/`	~3.8K	the OpenVAF `.osdi` bridge (a device family)

The single most surprising number: most of ngspice is device models, not "the simulator." The parser + analyses + solver together are a small fraction of the tree; the compact-model equations dominate.

Chapter 3 — The numerical background you'll need

Nodal analysis and the ”stamp”. Each component knows how to add its contribution to the system. A resistor `R` between nodes `a` and `b` adds conductance `g = 1/R` to matrix entries `(a,a)` and `(b,b)`, and $-$`g` to `(a,b)` and `(b,a)`. That four-entry pattern is its stamp. A component's `DEVload` routine is literally ”compute my currents and derivatives at the present voltages, and stamp them into the matrix and right-hand side.”

The G + sC structure. Reactive elements (capacitors, inductors) contribute a term proportional to the rate of change. In the frequency domain a capacitor stamps `sC` (with `s = jω`); in time it stamps a conductance-like term scaled by the integration coefficient. So the assembled matrix is really `G + sC` (or, in transient, `G + (a/Δt)·C`). ngspice keeps the resistive part (`G`, ”resist”) and the reactive part (`C`, ”react”) conceptually separate — a distinction that becomes explicit in the OSDI ABI (Chapter 12).

Newton's method. Nonlinear devices linearize about the current guess. One Newton step solves `J·Δx = −F(x)`, sets `x ← x + Δx`, and repeats. It converges quadratically *near* a solution but can overshoot from a poor start — which is why SPICE surrounds it with damping/limiting (per-device junction-voltage limiting) and homotopy (gmin stepping, source stepping). ngspice's convergence test is *iterate-based* (`|Δx| < reltol·|x| + abstol`); it has no native residual norm $\|F\|$ — a gap Enhancement-111 filled with an optional line search.

Sparse LU. The matrix is large but mostly zero (each node touches only its neighbors), so ngspice stores it sparsely and factors it with LU decomposition. Two solvers exist: Sparse 1.3 (the SPICE3 solver, with dynamic Markowitz pivoting) and KLU (SuiteSparse, faster on large static-pattern matrices). See the KLU vs Sparse 1.3 solver notes for how they differ in this build.

Integration. Transient analysis discretizes `C·dv/dt`. ngspice uses trapezoidal or variable-order Gear integration; the coefficients come from the recent timestep history. If the local truncation error (LTE) at a proposed point is too large, the step is rejected and the timestep shrinks.

Chapter 4 — The frontend: the command shell and control language

The frontend is a genuine interactive environment, historically called ”cp” (the command processor).

The command loop. Input lines are dispatched by looking up the first word in `cp_coms[]`, the master command table in `commands.c` — 239 commands, each a `{ name, function, … }` record mapping to a `com_*()` handler. The analyses (`op`, `tran`, `ac`, `dc`, `noise`, `pz`, `sens`, `sp`, `pss`, `tf`, `disto`) are commands; so are data operations (`let`, `print`, `plot`, `wrdata`, `fft`, `meas`), circuit management (`source`, `run`, `reset`, `alter`), and this project's additions (`pyplot`, and `osdi`/`codemodel` for loading models).

The **`.control`** language. A `.control … .endc` block is not just a script — it is a small structured language. `control.c` parses `while`, `dowhile`, `repeat`, `foreach`, `if`/`else`, and `begin`/`end` with labels and `goto`, building a tree of `struct control` (`co_children`, `co_next`) that `doblock()` walks and executes. This is why you can write real control flow — sweeps, convergence loops, parameter studies — around `run`.

Expressions and vectors. `let out = v(3)*2` and `print mag(vout)` evaluate arithmetic over data vectors (Chapter 14). The expression parser and the function library (`mag`, `db`, `real`, `imag`, `deriv`, `integ`, …) make the shell a small numeric workbench sitting directly on the simulation output.

Chapter 5 — From netlist text to a circuit

Turning deck text into a `CKTcircuit` is a pipeline, not a single parse.

1. Preprocessing — `inpcom.c`, at ~10,000 lines the largest single file in the frontend. It reads the raw deck and handles everything *lexical and textual* before any circuit exists: `.include`/`.lib`, line continuations (+), unit suffixes (`1k`, `1u`), `.param` substitution, the `.if`/`.elseif` netlist preprocessor, `.subckt` collection, hierarchical `.param` scoping, and B-source/behavioral rewrites. Several of this project's enhancements live here (e.g. the `` `__FILE__ `` / `` `__LINE__ `` textual pre-pass in E-85, the legacy `generate` pre-pass in E-88).

2. Subcircuit expansion — `subckt.c` flattens the hierarchy: each `X…` instance of a `.subckt` is cloned with its nodes and parameters renamed, so the engine only ever sees a flat circuit.

3. The three-pass parse — `spicelib/parser/`, the classic SPICE3 structure:

- Pass 1 (`inppas1.c`) keys off the first character of each card (SPICE2 convention: `R` resistor, `C` capacitor, `Q` BJT, `M` MOSFET, `.` dot-card …) and processes `.model` cards.
- Pass 2 (`inppas2.c`) creates the device instances and their nodes (calling into the engine via `INPgetMod`, `if_newnode`, and each device's `DEVparam`).
- Pass 3 (`inppas3.c`) resolves what is left, including device-internal nodes.

Behavioral expressions get their own parser: `inpptree.c` / `inpptree_parser.c` build a parse tree for the arbitrary expressions in B-sources and nonlinear E/G sources, and `inpeval.c` evaluates `.param` arithmetic. Dot-cards (`.tran`, `.ac`, `.option`, …) are dispatched in `inp2dot.c`.

The output of all this is a fully built `CKTcircuit` with a job queued for each requested analysis.

Chapter 6 — The seam: **`IFsimulator`** / **`IFfrontEnd`**

Between shell and engine sits a pair of function-pointer tables:

- **`IFsimulator`** — what the engine offers the front end: parse a deck, run a job, set a parameter, query a device, list the analyses and their parameters. The engine fills this in at startup (`SIMinit` in `main.c`).
- **`IFfrontEnd`** — what the front end offers the engine: allocate output vectors, emit data points, report errors, test for interrupts (`OUTpBeginPlot`, `OUTpData`, `IFerror`, `IFpauseTest`).

The concrete engine entry points are the `if_*` functions in `main.c`: `if_inpdeck` (text → circuit), `if_run` (run an analysis), `if_setparam` / `if_setparam_model` (alter), `if_option` (`.option`), `if_cktfree` (tear-down). Because everything crosses this boundary as data, the *same* engine is driven unchanged by the interactive shell and by `sharedspice.c` — the `ngSpice_Command` / `ngSpice_Circ` API of `libngspice`, used by Python bindings and GUIs.

Chapter 7 — The circuit in memory: **`CKTcircuit`**

Everything about a loaded, running circuit lives in one big struct, `CKTcircuit` (`cktdefs.h`). The fields you meet again and again:

- **`CKThead[]`** — an array indexed by device *type*; each entry is the head of a linked list of that type's models and instances. `CKTload` walks these lists.
- **`CKTmatrix`** — the sparse Jacobian (`SMPmatrix`, Chapter 9).
- **`CKTrhs`**, **`CKTrhsOld`**, **`CKTrhsSpare`**, **`CKTrhsOp`** — right-hand-side / solution vectors: the one being loaded, the previous (for the convergence test), a scratch, and the saved operating point. These vectors are 1-based (index 0 is the grounded reference).
- **`CKTstates[]`** — the state table. `CKTstate0` (a `#define` for `CKTstates[0]`) holds each device instance's per-point state: capacitor charges, inductor fluxes, and the *previous junction voltages used by limiting*. Integration reads older `CKTstates[k]`. This is why the E-111 line search had to make its trial re-loads *state-neutral* — limiting references live here.
- **`CKTmode`** — a bitmask describing what kind of evaluation is in progress: `MODEDC`, `MODETRAN`, `MODEAC`, plus init sub-modes `MODEINITJCT`, `MODEINITFIX`, `MODEINITFLOAT`, `MODEINITTRAN`, `MODEINITPRED`, `MODEINITSMSIG`, and `MODEUIC`. Devices and the Newton loop branch on these.
- **`CKTtime`**, **`CKTdelta`**, **`CKTdeltaOld[7]`**, **`CKTorder`** — transient time control: current time, next step, the seven most recent steps (for variable-order Gear), and the integration order.
- **`CKTcurJob`** — the analysis currently running.

Nodes themselves are `CKTnode` records (name, number, type). A node's *number* is its index into the RHS/matrix.

Chapter 8 — The device interface: **SPICEdev** and the registry

This is the abstraction that makes ngspice extensible. Every device family — resistor, BJT, BSIM4, an OSDI model, an XSPICE code model — implements one vtable of function pointers, `SPICEdev` (`devdefs.h:50`). The load-bearing members:

Member	Called when	Does
`DEVparam`	parsing	apply an instance/model parameter
`DEVsetup`	after parse	allocate this device's matrix entries (via `SMPmakeElt`) and state slots
`DEVtemperature`	temperature set	recompute temperature-dependent quantities
`DEVload`	every Newton iteration	evaluate at the present voltages; stamp conductances into the matrix and currents into the RHS
`DEVacLoad`	AC	stamp the complex small-signal matrix
`DEVpzLoad`	pole-zero	stamp with a complex `s`
`DEVnoise`	noise	add this device's noise contributions
`DEVconvTest`	convergence	device-specific convergence check
`DEVbindCSC` / `DEVbindCSCComplex`	KLU setup	bind the device's matrix pointers to the KLU CSC storage

The registry. `DEVices[]` (built in `dev.c` from a static list of `get_<device>_info()` functions) is the table of all known device vtables. The elegant part is that it is *not* fixed at compile time: `add_device()` and `osdi_add_device()` append new entries at runtime. When you run `osdi model.osdi`, each descriptor in the file is registered as a new **`SPICEdev`** at `DEVices[DEVNUM+i]` whose members point at the OSDI bridge functions (Chapter 12). XSPICE code models (`codemodel file.cm`) register the same way. So an OpenVAF-compiled transistor and a built-in BSIM4 are *the same kind of thing* to the engine.

Chapter 9 — The matrix and the solver: SMP → Sparse 1.3 / KLU

Devices never talk to a specific solver. They talk to the SMP ("sparse matrix package") interface (`sm-pdefs.h`, implemented in `maths/`), and SMP dispatches to whichever backend is active. The `SMPmatrix` struct (`smpdefs.h:27`) carries a flag `CKTkluMODE` selecting the path.

The key SMP calls:

- `SMPmakeElt(matrix, row, col)` — reserve a matrix entry, returning a pointer the device stores and later writes through. (Under KLU this appends to a COO list that is converted to CSC once the structure is complete; see Chapter 12 of the linesearch/KLU work.)
- `SMPaddElt` / the stored pointer — write a value during load.
- `SMPluFac` / `SMPcLUfac` — LU-factor the (real / complex) matrix.
- `SMPsolve` / `SMPcSolve` — forward/back-substitute for the solution.
- `SMPcaSolve` — the complex adjoint (transposed) solve used by noise and S-parameters ($A^T \cdot x = e$).
- `SMPmultiply` — sparse matrix-vector product (used by the E-111 residual merit).

Two backends. Sparse 1.3 (`maths/sparse/`) is the SPICE3 solver with dynamic Markowitz threshold pivoting — it re-orders on every factorization, which makes it robust on hard/degenerate matrices. KLU (`maths/klu/`, wrapped in `klusmp.c`) computes a fill-reducing symbolic ordering once and re-factors numerically thereafter — faster, but that fixed ordering is why balanced-output pole-zero stays Sparse-only (distortion, `.disto`, is Sparse-only for a different reason — it simply has no KLU binding). In this build Sparse 1.3 is the default; `.option klu` selects KLU. Details and the correctness map are in the solver notes.

Chapter 10 — The Newton core: **`CKTload`**, **`NIiter`**, convergence

Two functions are the beating heart of the analog engine.

`CKTload` (`cktload.c`): clear the matrix and RHS, then walk `CKThead[]` and call every device's `DEVload`. When it returns, the matrix holds the Jacobian `J` and the RHS holds `b`, both evaluated at the current guess. (The KCL residual is then exactly `F = J·x − b`.)

`NIiter` (`niiter.c`): the Newton loop. Each pass calls `CKTload`, factors the matrix (`SMPluFac`), solves (`SMPsolve`), and runs the convergence test (`NIconvTest`); it applies per-device limiting through the mode flags and, if enabled, the E-111 Armijo line search. It repeats until convergence or the iteration limit. `NIiter` is also where gmin/source-stepping homotopy hooks in when a straight solve fails.

`CKTop` (`cktop.c`): drives `NIiter` through the DC operating-point state machine — `MODEINITJCT` → `MODEINITFIX` → `MODEINITFLOAT` — plus the homotopy cascade (`dynamic_gmin` → `new_gmin` → `spice3_gmin` → `source stepping`) if the plain solve diverges. The residual is only a consistent function of the unknowns in the final `MODEINITFLOAT` phase — a subtlety the E-111 line search had to respect.

Convergence and limiting. ngspice's test is on the *iterate* ($|\Delta x|$ small), not a residual norm. Robustness comes from junction limiting (each diode/BJT clamps how far its junction voltage may move per step, referenced to the value stored in `CKTstate0`), node damping, and the homotopy cascade above.

Chapter 11 — The analyses

Each analysis is a driver in `spicelib/analysis/` that sets up a job, arranges the matrix/RHS, calls the Newton or complex-solve core, and streams results out through the front end.

- Operating point (**`.op`**) — one `CKTop`.
- DC sweep (**`.dc`**) — step a source/parameter, `CKTop` at each point.
- Transient (**`.tran`**) — `dctran.c`. This is the canonical time loop: seed with `CKTop`, then repeatedly predict the next point (`NIpred`), load + Newton-solve (`CKTload`/`NIiter`), estimate the local truncation error (`CKTtrunc`) to choose the next `CKTdelta`, and accept or reject the point. Variable-order Gear uses the `CKTdeltaOld[7]` history; the integration coefficients are computed in `nicomcof.c`.
- AC (**`.ac`**) — linearize about the operating point, then at each frequency build the complex `G + jωC` matrix and solve with `NIacIter` (`niaciter.c`).
- Noise (**`.noise`**) — the adjoint method: solve the *transposed* system once per frequency (`SMPcaSolve`), then sum each device's noise through it. This is why noise depends on a correct transposed solve — the bug fixed in E-113.
- Pole-zero (**`.pz`**) — find the `s` where the system determinant vanishes, via a Müller-style root search (`nipzmeth.c`) over `SMPcDProd` (the complex determinant).
- Sensitivity / S-parameters / distortion / PSS — the remaining drivers (`sensetup`/`cktsens`, `span.c`, `distoan.c`, `pss` under `--enable-pss`), several of which also use the adjoint solve. Sensitivity builds an auxiliary Sparse perturbation matrix `delta_Y` ($\partial Y/\partial p$) that is only multiplied, never factored — the KLU-safety of that split is the E-114 fix.

Chapter 12 — The OSDI bridge: how OpenVAF models become devices

This is where this repository's two halves meet. An `.osdi` file is a compiled shared library exporting a table of descriptors, one per Verilog-A module. The bridge in `src/osdi/` — only ~3,800 lines — makes each descriptor look like a `SPICEdev`.

Registration. `osdi model.osdi` (`osdiregistry.c`) `dlopen`s the file, reads its descriptors, and calls `osdi_add_device` to append a `SPICEdev` to `DEVices[]`. `osdiinit.c` wires the vtable: `DEVsetup = OSDIsetup`, `DEVload = OSDIload`, `DEVacLoad = OSDIacLoad`, `DEVnoise = OSDInoise`, and so on.

The descriptor ABI (`OsdiDescriptor`, `osdi.h:173`). The compiled model exposes:

- `setup_model` / `setup_instance` — one-time initialization.
- `eval(handle, inst, model, sim_info)` — evaluate residuals and Jacobian. The **`sim_info.flags`** field selects *what* to compute for the current analysis: `CALC_RESIST_JACOBIAN` (the conductance part `G`), `CALC_REACT_JACOBIAN` / `CALC_REACT_RESIDUAL` (the charge/capacitance part `C`, scaled by `alpha`), `ANALYSIS_DC | AC | TRAN | NOISE`, `ENABLE_LIM` (limiting), and more. One compiled model serves every analysis type through these flags.
- `load_jacobian_resist / _react / _tran`, `load_spice_rhs_dc / _tran`, `load_residual_resist / _react`, `load_noise` — copy the freshly-computed numbers into ngspice's matrix and RHS via pointers pre-allocated by `OSDIsetup`.
- `access(inst, model, id, flags)` — fetch a parameter or operating-point variable by id.

The load flow. `OSDIsetup` walks the descriptor's node/Jacobian layout and `SMPmakeElt`s every matrix entry, storing the pointers at fixed offsets inside the instance struct. `OSDIload` then, on each Newton iteration, assembles a `sim_info` with the mode flags, calls `descr->eval` (the OpenVAF-generated code computes residuals + Jacobian into the instance), and calls the `load_*` functions to stamp them. The resist/react split is exactly the MNA **`G + sC`** — the same distinction from Chapter 3, now an ABI contract.

Companion files handle the corners: `osditrunc.c` (the `$discontinuity` / `bound_step` timestep clamp, E-24), `osdiaccept.c` (per-accepted-point callbacks and deferred `$finish`/`$stop` requests, E-55), `osdiacld.c` / `osdinoise.c` / `osdipzld.c` (AC / noise / pole-zero loads), and `osdicallbacks.c` (the callbacks the model may call back into — `$simparam`, `$temperature`, message printing).

Chapter 13 — XSPICE: the event-driven engine and code models

XSPICE (`src/xspice/`) is a second engine bolted alongside the analog one, for digital and mixed-signal parts.

- Code models (`cm/`, built into `.cm` shared libraries like `analog.cm`, `digital.cm`) are C-coded behavioral models with their own `cfunc`/`ifspec` interface. `codemodel file.cm` loads them, and — like OSDI — they register as devices.
- The event engine (`xspice/evt/`) is genuinely different from the analog core: it is event-driven, not matrix-based. `evtqueue.c` maintains a queue of scheduled node changes; `evtiter.c` processes them; `evtnext_time.c` decides the next event time. Digital nodes carry discrete states, not continuous voltages, and are propagated by events rather than solved in a matrix.
- Bridges (`adc_bridge`, `dac_bridge`) connect the analog and event worlds so a mixed-signal netlist can co-simulate.

A netlist can therefore run two coupled simulators at once: the analog Newton/matrix loop and the XSPICE event queue, synchronized in time.

(A third numerical path, CIDER in `ciderlib/`, solves physical device equations on a spatial mesh — TCAD-style. It is niche and mostly independent of the flow above.)

Chapter 14 — The output data model: dvecs, plots, the mini-MATLAB

Simulation results are not files; they are in-memory vectors the shell can compute on.

- A **`struct dvec`** (`dvec.h`) is one data vector: `v_realdata` or `v_compdata` (real or complex array), `v_length`, a `v_type` (voltage, current, time, frequency…), and a `v_scale` (its independent axis).
- A **`struct plot`** (`plot.h`) is a named result set — `tran1`, `ac1`, `op1` — holding a linked list of its dvecs (`pl_dvecs`) and the shared scale (`pl_scale`). Plots form a list; the "current plot" is what unqualified vector names resolve against.

When an analysis runs, the engine calls `OUTpBeginPlot` (create the plot) and streams points with `OUTpData` (append to the dvecs) — `outitf.c` is the engine→frontend data path. Afterwards, `let/print/plot/fft/meas/wrdata` all operate on these dvecs. That is what makes the `ngspice>` prompt a small numeric workbench: the results are live vectors, and the command language is a calculator over them.

Chapter 15 — A complete worked example: an RC circuit end to end

Take the simplest transient run:

```
* rc
V1 in 0 PULSE(0 1 0 1n 1n 1u 2u)
R1 in out 1k
C1 out 0 1n
.tran 1n 5u
.end
```

1. Read & preprocess. `inpcom.c` strips comments, joins continuations, expands nothing (no subckts/params here), and hands three device cards and one dot-card to the parser.

2. Parse. Pass 1 sees no `.model`. Pass 2 keys off `V`/`R`/`C`: it creates a voltage-source instance (nodes `in`,`0`), a resistor (`in`,`out`), a capacitor (`out`,`0`), allocating nodes `in`, `out` (node `0` is ground). `inp2dot.c` queues a transient job from `.tran 1n 5u`.

3. Setup. Each device's `DEVsetup` calls `SMPmakeElt` for its stamps: the resistor reserves `(in,in)`, `(in,out)`, `(out,in)`, `(out,out)`; the capacitor reserves `(out,out)` and a state slot for its charge; the source reserves its branch. The matrix structure is now fixed (and, under KLU, converted to CSC).

4. Operating point. `DCtran` first calls `CKTop`. With the pulse at 0 V, the solution is trivially `v(in)=v(out)=0`; `NIiter` converges in one step. This seeds `CKTstate0` (the capacitor's initial charge).

5. The time loop. For each step, `dctran.c`:

- `NIpred` extrapolates a guess for `v(out)` at the new time;
- `CKTload` stamps `G` (the resistor's `1/1k`) and the capacitor's `C/`Δ`t`-scaled companion conductance + current into the matrix/RHS, using the integration coefficients from `nicomcof.c`;
- `NIiter` solves (this circuit is linear, so one iteration);
- `CKTtrunc` estimates the LTE from the charge history and picks the next `CKTdelta` — small during the 1 ns pulse edges, larger on the flat `RC` $\approx$ `1` μ`s` tails;
- `CKTaccept` commits the point, rolls `CKTstates`, and `OUTpData` appends `v(in)`, `v(out)` to the `tran1` plot.

6. Result. When `CKTtime` reaches 5 μs the loop ends. The `tran1` plot now holds `time`, `v(in)`, `v(out)` dvecs; `plot v(out)` shows the RC exponential edges. Swap `R1`/`C1` for `N1 in out mymodel` with a `.model mymodel …` backed by `pre_osdi mymodel.osdi`, and step 3's `DEVsetup`/step 5's `CKTload` route through `OSDIsetup`/`OSDIload` instead — everything else is identical.

Chapter 16 — Where this project touched ngspice

The enhancements in this repository land in specific corners of the map above — a useful index if you are tracing one:

Area (chapter)	Files	Enhancements
Netlist pre-processing (5)	`frontend/inpcom.c`, `inp2dot.c`	`__FILE__`/`__LINE__` (E-85), legacy `generate` (E-88), bare `generate` (E-96)
Analyses (11)	`spicelib/analysis/*`	`.dc @inst[param]` & `.tf`/`.pz`/`.sens` (E-62), RF `.sp`/PSS (E-63), Touchstone (E-64, E-72), sim-control tasks (E-55)
Newton core (10)	`maths/ni/niiter.c`	`.option linesearch` globalized Newton (E-111)
Matrix / solver (9)	`maths/KLU/klusmp.c`, `klu_multiply.c`, `spicelib/analysis/cktsens.c`	KLU line search (E-112), KLU noise + single-ended pole-zero (E-113), KLU sensitivity (E-114)
Options / tasks	`spicelib/analysis/cktsoptn.c`, `cktntask.c`	`.option errpreset` (E-110), `.option linesearch`/`.option klu` plumbing
Frontend com-mands (4)	`frontend/com_*`, `commands.c`	`pyplot` matplotlib plotting (E-94/95/98/99)
OSDI bridge (12)	`src/osdi/*`	`$discontinuity` clamp (E-24), `ac_stim` AC-RHS (E-51), noise factors (E-54), final-step phases (E-53), multi-module libs (E-76)
Build / lifecycle	tree-wide	zero-warning macOS/clang build (E-77), session lifecycle (E-81)

The ngspice change report has the file-by-file detail; the gap analysis places ngspice against a commercial simulator.

Chapter 17 — Reference appendices

A. Key data structures

Struct	Header	Role
`CKTcircuit`	`cktdefs.h`	the entire loaded circuit + simulation state
`CKTnode`	`cktdefs.h`	one circuit node (name, number, type)
`SPICEdev`	`devdefs.h`	a device family's function-pointer vtable
`GENmodel` / `GENinstance`	`gendefs.h`	the generic model/instance headers every device extends
`SMPmatrix`	`smpdefs.h`	the sparse matrix (dispatches Sparse 1.3 / KLU)
`OsdiDescriptor`	`osdi/osdi.h`	one OpenVAF module's ABI table
`IFsimulator` / `IFfrontEnd`	`ifsim.h`	the shell⇄engine seam
`dvec` / `plot`	`dvec.h` / `plot.h`	output data vectors and result sets

B. Key functions to start reading from

Function	File	What it does
`SIMinit`	`main.c`	publish the `IFsimulator` table
`if_inpdeck` / `if_run`	`main.c`	parse a deck / run a job
`INPpas1/2/3`	`spicelib/parser/inppas*.c`	the three parse passes
`CKTload`	`spicelib/analysis/cktload.c`	stamp every device into the matrix
`NIiter` / `NIacIter`	`maths/ni/niiter.c` / `niaciter.c`	real / complex solve loops
`CKTop`	`spicelib/analysis/cktop.c`	the operating-point state machine
`DCtran`	`spicelib/analysis/dctran.c`	the transient time loop
`OSDIsetup` / `OSDIload`	`src/osdi/osdisetup.c` / `osdiload.c`	the OpenVAF model bridge

C. **`CKTmode`** flags (grep these to follow analysis branching)

`MODEDC`, `MODETRAN`, `MODEAC`, `MODEDCOP`, `MODETRANOP`, `MODEDCTRANCURVE`, `MODEINITJCT`, `MODEINITFIX`, `MODEINITFLOAT`, `MODEINITTRAN`, `MODEINITPRED`, `MODEINITSMSIG`, `MODEUIC`, `MODEACNOISE`, `MODESP`, `MODESPNOISE`.

D. Glossary

- MNA — Modified Nodal Analysis: unknowns are node voltages (+ a few branch currents); equations are KCL.
- Stamp — a device's additive contribution to the matrix/RHS.
- Resist / react — the conductive (`G`) and charge/reactive (`C`) parts of a stamp; together they form `G + sC`.
- State table (**`CKTstate0`**) — per-instance stored charges/fluxes and limiting references.
- SMP — the sparse-matrix interface layer; dispatches Sparse 1.3 or KLU.
- Adjoint solve — the transposed solve ($A^T x = e$) used by noise/S-parameters.
- OSDI — Open Source Device Interface: the ABI by which compiled `.osdi` models plug in as devices.
- Code model / **`.cm`** — an XSPICE C-coded behavioral model.

Closing note

The shape to keep in your head: a shell talks to an engine across a thin seam; the engine turns a netlist into a `CKTcircuit`, and repeatedly asks every device to stamp a sparse matrix that a solver factors — that inner loop, wrapped in Newton's method and an integration rule, is the whole simulator. OSDI and XSPICE are two doors in the device wall through which compiled models walk in as first-class citizens; the OpenVAF half of this project simply makes very good use of the first door.